Algorithms · P1 to P6

These programs are examples from the notes. To understand them, state the inputs, the output and the assumptions, then carry out a few steps by hand. The answers to the activities and homework remain to be written up separately.

P1 · Terms, sum and threshold

Suppose n is a non-negative integer and the safety cap a positive integer. The first function returns u_n and $\sum_{k=0}^{n} u_k$ for $u_{k+1} = au_k + b$. The second seeks a first upward crossing; its safety cap only signals that it was not found within the set number of steps.

```
def terme_somme(n, u0, a, b):
    u, total = u0, u0
    for k in range(n):
        u = a*u + b
        total += u
    return u, total

def seuil_croissant(u0, a, b, cible, maximum=100000):
    u, n = u0, 0
    while u <= cible and n < maximum:
        u = a*u + b
        n += 1
    if u <= cible:
        return None
    return n
```

P2 · Bisection

The calculations below use floating-point numbers; the mathematical guarantee assumes the signs correctly determined. Assumptions: f continuous on $[a, b]$ and $f(a)f(b) \leq 0$. To identify a unique zero, prove uniqueness separately. The function returns an interval of length at most ε, as long as machine precision still allows the interval to be split.

```
def dichotomie(f, a, b, epsilon):
    assert a < b and epsilon > 0
    fa, fb = f(a), f(b)
    assert fa*fb <= 0
    if fa == 0: return a, a
    if fb == 0: return b, b
    while b-a > epsilon:
        m = (a+b)/2
        if m == a or m == b:
            raise ArithmeticError("machine precision reached")
        fm = f(m)
        if fm == 0: return m, m
        if fa*fm < 0:
            b, fb = m, fm
        else:
            a, fa = m, fm
    return a, b
```

P3 · Euler

With N a strictly positive integer and $t_0 < \text{fin}$, for $y' = F(t,y)$, $Y_{k+1} = Y_k + hF(t_k, Y_k)$. The program returns all approximations; it does not prove their accuracy.

```
def euler(F, t0, y0, fin, N):
    assert N > 0 and fin > t0
    h = (fin-t0)/N
    y, valeurs = y0, [(t0, y0)]
    for k in range (N):
        t = t0+k*h
        y = y+h*F(t, y)
        valeurs.append((t0+(k+1)*h, y))
    return valeurs
```

P4 · Rectangles

For N a strictly positive integer and f continuous and increasing on $[a,b]$, the result gives a lower bound and an upper bound for the integral. Without monotonicity, these are only two approximations to examine.

```
def rectangles(f, a, b, N):
    assert N > 0 and a < b
    h = (b-a)/N
    gauche = h* sum(f(a+k*h) for k in range (N))
    droite = h* sum(f(a+k*h) for k in range (1, N+1))
    return gauche, droite
```

P5 · Binomial distribution and simulation

Suppose n is a non-negative integer, $0 \leq p \leq 1$ and k is an integer. The direct calculation below is suitable for the modest sizes used in the exercises; for very large parameters, specialised numerical libraries are preferable.

```
from math import comb
from random import Random

def probabilite(n, p, k):
    if k < 0 or k > n: return 0.0
    return comb(n, k)*p**k*(1-p)**(n-k)

def cumul(n, p, k):
    return sum(probabilite(n, p, j)
               for j in range(min(k, n)+1))

def simuler_binomiale(n, p, generateur):
    return sum(generateur.random() < p for _ in range(n))

rng = Random(2026)
```

P6 · Fluctuations of a frequency

Suppose N is a strictly positive integer and $0 \leq p \leq 1$. The seed reproduces the same example. Repeat with other seeds; the law of large numbers does not say that the observed deviation decreases at each step.

```
from random import Random

def frequences(N, p, graine=2026):
    rng = Random(graine)
    succes, resultat = 0, []
    for n in range(1, N+1):
        succes += rng.random() < p
        resultat.append(succes/n)
    return resultat
```